Task 2 – Datasheet Literacy

Datasheet-Based Design Analysis: L298N Motor Driver

1. Datasheet Parameters and Their Influence on Circuit Design

The L298N is a dual full-bridge motor driver used to control the direction and speed of two DC motors.(in my case and posibly 4 also)

a) Motor supply voltage

4.5–13.5 V.. this is the voltage suplied to the motors and this helped me choose the battrey for it and also the motors too...

b) Output current rating

1.2 A continuous per channel, 3.2 A peak. And also the peak current of a n20 motor is 0.8 A which is very suitable .. to use the above driver ..

c) Logic input voltage

2.7–5.5 V, with a separate motor supply, VCC covers both 3.3 V and 5 V logic, so I wired it directly to my microcontroller's logic rail with no level shifter. VM and VCC are separate pins on the module, so the motors run from the battery while the logic stays on a clean regulated rail. The motor supply never feeds the microcontroller pins (main advantage ..

2. Condition That Could Damage the Component

Exceeding the 15 V absolute maximum on VM. Above this, the output MOSFETs and internal circuitry can break down, and the damage is permanent. In practice the risk comes from a wrong battery (for example 4S Li-ion at 16.8 V), voltage spikes from the motor, or reversed polarity. The breakout module has no reverse-polarity protection, so reversed battery leads would also destroy it.

To reduce this risk, the motors' current requirements should be checked before use, including their stall current. The driver should be operated within its electrical and thermal ratings. The motor supply voltage must also remain within the permitted range.

How my design avoid the problems are that i will first use a 3A fuse .. and also im using a 7.4 v Lipo battery and buck converter to make sure no wrong volatge go into the motor driver ... and also one very very main advantage is using a buck converter makes sure we get a steady supply og 6v to the driver .. so steady voltage to the motors so having more control on the car caz we are using PID

Task 3 – PID Control system

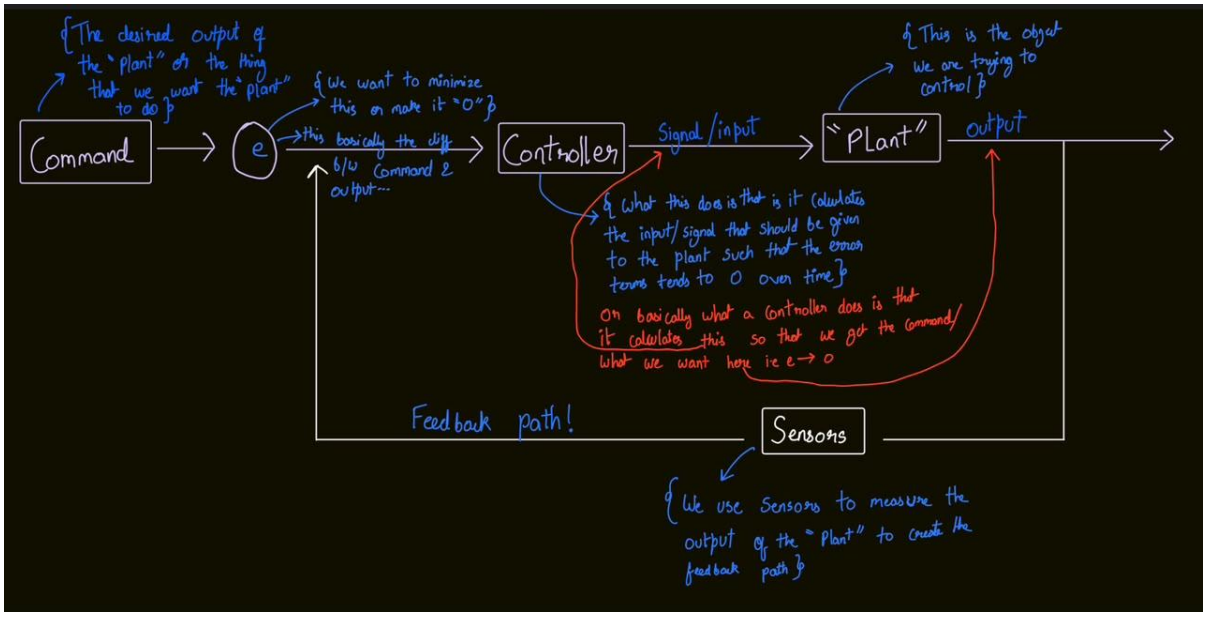
To understand what PID is we will first understand what a control system is …

Control System – It is a system that is designed to make our machine behave in our desired manner

- There are 2 type of control systems-
  - Open Loop Control System – Suppose a fan .. that runs for our desired time and switches off after the set time .. i.e basically a open loop control system does not change the inputs of the "plant" by/after checking its output.. I.e the fan does not check if it the desired result is achiefed or not .. it runs for the set time and turns off even if the room is still hot
  - Closed Loop Control System – This system contoinuously checks the output of the "plan" and keeps changing the input to the plant so that our desired result is achieved Say a smart ac which keeps measuring the room temperature and reduces the cooling as the room keeps getting cooler and finaly stops after reaching the desired temperature
  - So basically if the input is adjusted corresponding to the output to get the desired result it is called a closed loop
- We use sensors and a controller to change the input so that we get a desired output

- One Such controller in a closed loop control systems is PID

PID Controller

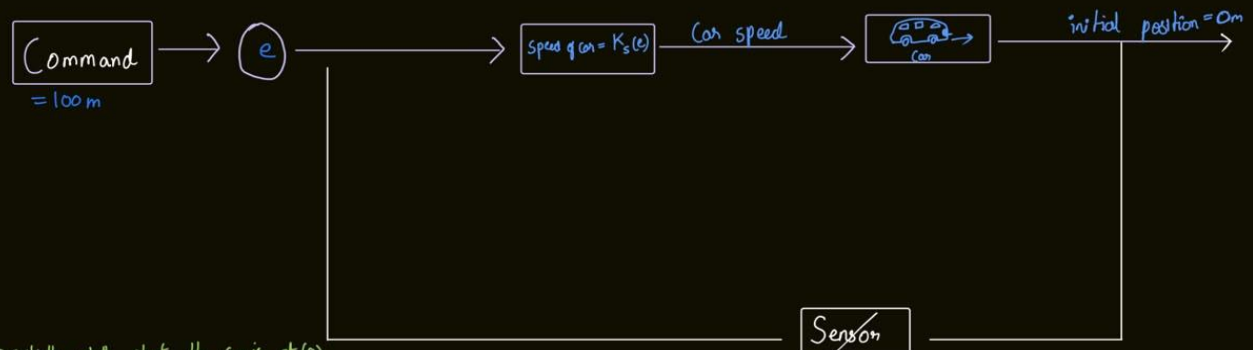
- Read the below so you can understand well (my handwritten stuff)
- 

6:32 PM Mon 26 Oct 32%

Linear algebra 1 | Untitled

Lets try to understand this by another example where the controller is a simple proportional controller..

Say we have a Car that has to travel 100 m ...



1) initially at the start the car is at (0)
So the error term (e) = 100 - 0 = 100 m

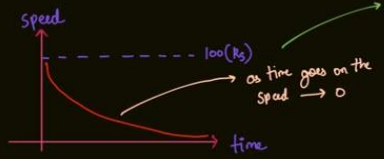
2) Since the e = 100 the controller tells the car to move at speed = K_s(100) Say K_s = (0.1) then S = 10 m/s

3) Say after some time the position of car moves closer to the 100 m so e term decreases & the speed also keeps decreasing

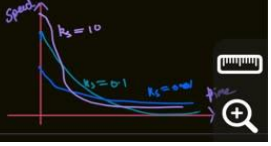
4) lets try to visualise this by some graphs..

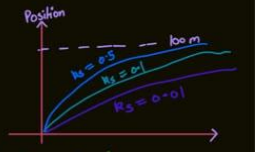
Speed = K_s (error)
error = 100 - (position)





We can basically change the K_s to change how quick / slow we need to go the 100 m...





6:32 PM Mon 26 Oct 32%

Linear algebra 1 | Untitled

Now Why PID & why not just proportional controllers..

→ So we saw that the proportional controller could do the task above & now lets discuss PID.

→ Suppose the example of a Cruise control car... & here

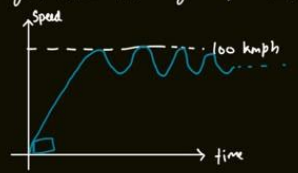
① Command = 100 km/h speed
② input given to the motors is say in RPM
③ Output is the speed of the car...
④ Sensor is the speedometer...

Case - I (Just proportional controller)

→ here say the car was at rest, the errors here will be max & the rpm of the motors will also be high...

→ So as the car speed gets closer and closer to 100 the errors gets closer to 0 & then rpm of the motor gets to 0 & so does the speed of the car..

→ now either of 2 thing happen... Say it reaches the 100 kmph, now the e becomes 0 so the, controller makes rpm 0 & the car stards to slow & e again starts increasing & again reaches 100 ... So on.....



→ So what happens is it reaches 100 but does not stay at that speed for long time...

→ it can also so happen that it never reaches 100 kmph

Say 1000 rpm = for the car to cruise at 50 kmph..

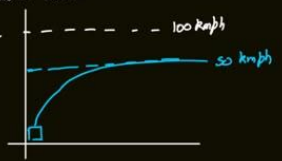
→ Say k_p = 20 then

rpm = 20 (e)

initially e = 100 so rpm increases & so does speed & say like this speed keps on increases & e = 50 kmph or car reaches 50 kmph & at that speed

rpm = 20(50) = 1000 .. but 1000 rpm corresponds exactly to maintain 50 kmph ie it comes to 50 kmph & stays & can never increase...

→ this is called the steady state errors..



Case - II (P + I)

→ now to try solve the issue we add a integral term..

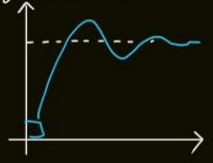
initially rpm = $K_p(e)$

now rpm = $K_p(e) + K_i \int e\,dt$

now what this does is it keeps adding all the errors terms..

→ Consider the same example but now with the integral term...

So say the car reaches speed = 100 but this time the motors dont stop cos the integral term is not 0 .. & so it overshoots .. & as it overshoots the e becomes -ve & integral also starts decreasing ... So it goes like this

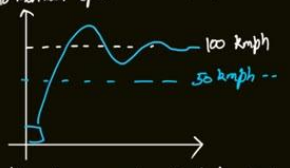


So in P+I controller Compared to the P controller there are say less "jerks" and reaches desired value more controlled manner.... (Which is highly important for time following robots..)

6:32 PM Mon 26 Oct — 32%

Linear algebra 1 — Untitled

→ Seems good right -- but there is still a tiny problem --- if you remember the steady error will also be solved because here even though $k_p(e) == 0$ the rpm req to maintain a speed but integral term is still +ve -- so total rpm > rpm req to maintain speed --- hence it will no longer be stuck at a speed to ....



→ basically if you visualise before coming constant speed = 100 km/h the speed of the car overshoots --- & the value of e becomes <0 & the integral term reducing ---

→ even though it still comes to const speed = 100 km/h but that overshoot may not be desirable --

→ now how do we fix this ?

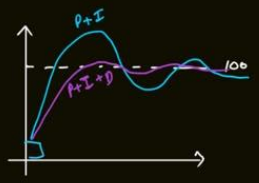
Case - III (P+I+D)

→ now what we do is we add a derivative term too so

$$rpm = k_p e + k_i \int e\,dt + k_d \frac{de}{dt} \rightarrow P+I+D$$

→ what this term does is it measures the change in e, say we are approaching speed = 100 so the e decreases & the $\frac{de}{dt} < 0$ so it tries to reduce the rpm --

→ So basically what the D term does is, it exactly reverses the effect of the integral, i.e to compensate the overshoot the derivative creates & undershoot so that the overall rpm comes to 100 more smoothly basically!



67% | 4 of 5

Task 4 – BOM

IR Sensor Array (QTR-8 RC)

- We can use a premade ir sensor array for this project instead of manually taking multiple ir sensor modules … basically
- It has the following advantages
    - In the premade ir array the most important advantage is that the 8 pairs come presoldered and with also equal spacing so first thing is we dont need to take the pain to measure the dinstaces and make sure all the 8 modules of ir sensors are fixed and also that there will be vey very less pain that we need to take during the calibration …
    - And also the connections will also be easy since we will get all the pins directly soldered to one singular board so the connections will be more organised and the connections will also not look that shabby
    - Link - https://www.amazon.in/Electron-QTR-8RC-Following-Reflectance-Sensor/dp/B0DYP4JP2V?mcid=8d4085bf35be3cfd955659775a985b49&tag=googleshopdes-21&linkCode=df0&hvadid=709963085510&hvpos&hvnetw=g&hvrand=11556278949578119663&hvpone&hvptwo&hvqmt&hvdev=c&hvdvcmdl&hvlocint&hvlocphy=9183877&hvtargid=pla-2422909312205&psc=1&hvocijid=11556278949578119663-B0DYP4JP2V-&hvexpln=0
    - Price – 240Rs

Microcontroller (ESP 32 WROOM 32 Devkit board)

- We can also use the chip itself .. but using the devkit board is more easy.. So devkit board it is
    - Link - https://www.amazon.in/SP-ELECTRON-ESP32-Development-ESP32-WROOM-32/dp/B0HD7QC26L?crid=2K6Z1AM6WO9VD&dib=eyJ2IjoiMSJ9.H5Qdvll8rdKi4WqCiS9VhdI3iaZbhgtQl5klI5fbAqTaHnH9PVfhgVGmASJWMuZNB3Mxu6roYG4ggagBCZ7XaCWCBcJvx_T-YgyaVu8gdqFZE-4fkuTlUJdxvgzD6RV8qxLBlogARd1CI1B5OTSbrFFtjrvCrieT-9f5GAYHtIQ6vPddJV-NWIbnqPjV12Tw1KI-4X_kkj5czKc-yjeG4fTURmEwLl-IIzfKoPTiqysDBVjEEEs0xjfvRMQ_3V4bB2NMqUA4Y4goWrmo1mJqeYysbLSQQruYxZcDzECG-SM.LJbMbiMqNHSWJKMeUsR8qRuBlAAKLQ8eyhUiBMuj1z0&dib_tag=se&keywords=esp+32+WROOM+32&qid=1792926887&s=industrial&sprefix=esp+32+wroom+3%2Cindustrial%2C385&sr=1-1
    - Price – 500Rs

Motors (2 N20 6v Metal Geared Motors{200rpm} with encoders)

- The main reason of me not using the 300 rpm motor is eventhough it is faster it is harder to tune ..and besides we are not making a line following robot where speed is the priority our priority is accuracy and easier tuning so i think the 200 rpm will do just fine
- And also compered to normal tt plasic geared motors .. the main difference is the gears (which are metal in n20) and also their operating voltages .. and also has an encoder .. to measure the roations of the motor .....
- Basically more control during turnings and intersections .. and far less pain during tuning..
- And the reason for the encoders is because we can map the maze ..
    - Link - https://www.amazon.in/200RPM-Micro-Metal-length-Encoder/dp/B0G49CC6YW?crid=3SMQO1HO4OMWI&dib=eyJ2IjoiMSJ9.IiFODmiFUn4Yi0w6gZSeuVWxcZkYJ94dEZewNoefHkrjb2bYAcOb9CeZA65lPkWxwO9VZsRXGqGKzzTBRmWnNWG_KQbTb4P9ei96Tvu59oFKAkUYg9mikC9RnTjmWncLVR-J90AWtkY4ILPZmjVmC9Qw5FfXPKLCo01mBh7klzYnrcsNNqNrUCVkkp4iEoSLWyaSXoIGLkwZiBwDuQUe47U8_HjukcBQ64aXYlXwMNqihOPqz2uqF6ul9sgEtPwVM0KUNM0Y2o3D3IFUy3bG0rhddLYJk5H5FWoKe0KPNt0.K8yGXiBtykzxvx-28_CDnresWipz3i5TDnyScKML3FM&dib_tag=se&keywords=n20+Geared+motors+200rpm&qid=1792927333&s=industrial&sprefix=n20+geared+motors+200rp%2Cindustrial%2C297&sr=1-1
    - Price 1200Rs
- For Wheels we need some wheels with more traction so something like these below...
    - Link - https://dyoraarobotics.in/product/2
    - Price 269Rs
    - Or we can get soemthing like this also - https://www.theengineerstore.in/products/n20-dc-gear-motor-wheel-42mm?variant=43720257994915&country=IN&currency=INR&utm_medium=product_sync&utm_source=google&utm_content=sag_organic&utm_campaign=sag_organic
    - Price 180Rs

Motor Driver (TB6612FNG Dual DC Motor Driver Module)

- The main reason we are using this is in line folowing bots generally .. we have rapid acceleration and suddeen breaks .. so to have better control we use the above(especially with pid controller)
- And also that the l298n is more bulky
    - Link - https://www.amazon.in/TB6612FNG-Dual-Motor-Driver-PACK/dp/B0GMVYN79G
    - Price – 400 Rs

Miscellaneous Stuff

1. LED Light
    a. For indicating that it finished the maze
    b. LED Toh club se mil hi jaayegi .. khareedne ki kya jaroorat 🥀🥀🥀

c. Link - https://www.amazon.in/Electronicspices-Ultra-Bright-White-Diode/dp/B085BMH3XZ?crid=1RMTZFUN22F45&dib=eyJ2IjoiMSJ9.-cwHRl7D3tkTZrkwgsrs1i33jua-2d8lJZRwI9BjtWLGP9ABlPwH14FOhKKCR6MuO3fQmsvU4sGltYJUJmyPzLZM9jK38sZmActQI_qFlMUIqKdrVHIHMFFXEbPPihfEoqqTIfrY_yinPvGXZcymY0wE8yc5rgmY6cWJVdBJzhoAEuNpfr8qTY4TcGxioZyBPH2rsypDF1YBz72x28HVcDoGMJ9_hbIIx8Yr7cb4CWRpoSdwUbvhTLkVSoaFMrxyupPJmOzO1FUoutb-GAmzl2HX4JfEcN9MQAjus1ui8Qk.dimrmvLh2qJJvHRXkAWKK5Me85e_ld65yIFTocHJjow&dib_tag=se&keywords=single+led+light&qid=1792956319&sprefix=single+led+ligh%2Caps%2C324&sr=8-7

d. Price – 50 Rs

2. A Switch
    a. This too .. club se hi utha lenge
    b. Link - https://www.amazon.in/SP-Illuminated-Electronics-Automotive-Appliance/dp/B0H6F78F97?crid=2I11LVZ0ZXP0V&dib=eyJ2IjoiMSJ9.7VsmIOwBhZtMOOwoROD4Bjy10WjQ5FzPBCHhzmdyUE6cMd8ftbH-x4OLg-ebYPnh6sf8isKFQFg4h1D-A_Dnizx9sy4EIqWXjzP978ocoG8jMRnQITYjNLv6H1vshoRQpmI4nYYaAY3oYaDV5O8MCxwXOIfV6Z6e9krBDPzyyOA_hjQ332DzMAGFyXUlDkkdg08v052mEobbHjn5KZoFAL_9qgDE8AInlkjpZaOfIUyjSaUCRX0Z44boyoUFcTwDDZA0nxcjtwxYmUBcHrF9fXlYpumci1YIr1l-lUO0K0Y.LFP7ZUaqg6tQw8_nfKyGr-sRr4yY7O2Nwdat6xRil6c&dib_tag=se&keywords=small+switch&qid=1792956360&sprefix=smallswithch%2Caps%2C379&sr=8-6

    c. Price – 30 Rs
3. Chasis
    a. We want it to be less bulky ..
    b. Something like below can work (My personal preference btw .. we can pick something else also ..)
    c. Link - https://www.amazon.in/Round-Acrylic-Robot-Chassis-Black/dp/B0F47K9N19?crid=6QN0RTTGZ8YM&dib=eyJ2IjoiMSJ9.856Ifh5W_4uj8KK69ib8xmKDPJgfXLcTyrZHXn6idEqs-x0SGUExU8W2lK3G4QBMDWQF16Tp7IRQs-Prsx9_vSrcQ227eSCYA87jGbkSxE0jejRvb5TOm2J7hjsgJIl_OCVjPiGDDZggFm81PFtcugPmqgUWAKtFfk8wi94JVDsOcCVWUMMnzOK3SnLUv-FnXt6uGSiwnX5Xd9u3yZGIfzMZnI7KDCx4bkdkL2mi3DSIHrHK6oWo7jJwpRaogC5Nxsw3NJnqOKoav7AyftORtuCEfm2HUui7phr1URfQtlM.4eW-1UESuptiZtQlDq627W3o_9XS9CsuDTufjrhRxdU&dib_tag=se&keywords=car+chassis&qid=1792956400&sprefix=car+chas%2Caps%2C382&sr=8-27

    d. Price – 600 Rs
4. Connector and zero pcb Board and jumper wires
    a. Link - https://www.amazon.in/UNIVERSAL-HUB-General-electronic-Experiment/dp/B0C3B8R4CX?crid=G20W320OA1I4&dib=eyJ2IjoiMSJ9.rvprrkAGoYDmTGGZcpW5-LB_SSX_qKavGwy7gbx-QGJF7kk0xkuVN1XKLsn3NYmE5XCfMSEE4FLf05Ft66lQPR0mwJN1HZngcRh31YkBpQ1sH0ZhXidhxUXohDT28-Ux4XShc341JAhvN7J5jAGsc3LcW7ViuYpdDdtCnIj8b6fFo8jx3hf4BvZofGQPE9tkjL0L3Ai7upn72W0uXyYBwrW4QPvwfK3dXA4_inPigjEg_I1Kftf6PdxaWCGeFMPDJWe9T_i

ycP7fzX8XEMrV8pmcKRtgRaxA3ttXQ4tWEWk.CCd9I4jpdmPiEeVshjx9g_u_gYI5TR9Bh1fYSFlxTaE&dib_tag=se&keywords=pcb&qid=1792956643&sprefix=pcb%2Caps%2C365&sr=8-6

b. **Link -** https://www.amazon.in/UNIVERSAL-HUB-2-54mm-Female-Connector/dp/B0DNMD4THF?crid=1J8VLXGGWHWP&dib=eyJ2IjoiMSJ9.NpiqG_GJTt7YLKhI8CJENxT9AvEhnhfPJXrrYLyOERHSGC4_kxLXNqnx0yyzk-aqy7g2cLjQEYlCnzHF__0pjGKnuop75452prkhQ4znJBnFSVYBcz9eldmtTRidzbQc4DmuNrfiCceRYMJNM6W7eL5pr_Xqyc6JkmTgwn8p1R1btbeRhk5E5Tm8h0gLLnbxJd7xVw9twV3TtPaPuX2JgTNDe5iHCTgdcy52Qe1m_cebNtl-nfkhR3N2oAA6v9CQbXuUWuf6_HwUrBcY5VvgNCYjBU1mlOp7oGow1_iabwk.z5YUZ65MlMrMsfVV9tS6T3RaRvppjT4B6pGOHPkhtww&dib_tag=se&keywords=connector+for+arduino&qid=1792956703&sprefix=connector+for+arduino%2Caps%2C336&sr=8-9

c. **Link -** https://www.amazon.in/Electronic-Spices-Single-Core-Breadboard-Jumper/dp/B0B4WPX449?crid=9X3VZ73B5WV3&dib=eyJ2IjoiMSJ9.vLBL377-brAFNacDz1hApdJzm2dtYtf3ydYQDIMqX9zDlBkcyG_U3TdCXK5vMR9_ddvE36Q8kyzVFZUWfawGi5HX2bVezUOWNIJ5wCM8SDvTGkSNGIQhMOUNQojrLdxHsjpogqVwPZ_fkp7bKSqgTcFXwIpiI6pCDEHFyLO1ZlHowV1UmZL_rZLIJaQ_HRltK4aQQLb7xKn7bq_OYg5ZD2vpYRjQvhvW7r4lnTxXE0EonLFN2yQLBu_zELQ38c8Refl21fFhmgnWo14Cw1M5T8ozV3_z9JL4XUzh_y-4SoI.2GsUc3g-NHeFeNYO9lT6TlBktI4g_z5y3Y-cKrtoV-k&dib_tag=se&keywords=wire&qid=1792956733&sprefix=wire+f%2Caps%2C351&sr=8-10

d. **Link -** https://www.amazon.in/Aptechdeals-Jumper-Wires-Female-Breadboard/dp/B074JB6SX8?crid=U5C7KWHH5QD1&dib=eyJ2IjoiMSJ9.NpiqG_GJTt7YLKhI8CJEN93BeTCKz0zXicFcYBnJD_L-pdUlzupjXnJ_2TqtdUJsGJk-8IcymmMln_8F8uswHeH1QdxGKFiKcD2USV5Tu0hDG8Fz0M_p8cqT91tMxepvU5dn19dCKcgfS0MdT-XWe8XavWfLxDYEtzJEIXGPG3RhQjtzZrpBLPuYUnEfzla6d5Q9cyMx2Wzes7IyJzkFr9NUYjIX5ypTQE4U6F_txL8FZu03rW8NE70vkxjWNH8I8ZmP3-VhhLXpIJgnOdwythShLkztNNB1MVGuvo5PlAI.9e1AUkH8El0hvo9vWiJb-emdn0oyzJONPfReXiNrTzw&dib_tag=se&keywords=jumper+wire+set&qid=1792956577&sprefix=jumper+wire+se%2Caps%2C353&sr=8-6

e. **Price – 60 + 100 + 150 + 140 = 450 Rs**

5. **Buck converters ..**
   a. **We need 2 actually one for the 5v and one for the 6v**
   b. **Link -** https://www.amazon.in/Robocraze-Lithium-Battery-Charging-Converter/dp/B07RKH7YVY?crid=1XH81LRF6O6IS&dib=eyJ2IjoiMSJ9.SdP7Hn0UC0hg9sVzZKNKRjp7VQxWtL7Ptks-qowE_IkszxkIlcpUC_XSiOTL10xQI-ea_cagnk7hvw7Dg_b255KHExF-udaxGtXKvYp0t5xakESHLX_v5TA2SA71ODqupyO4HrLPWOgsOUai85we0WdIGsTBUcaXxRRtjEBLO39qWgWPe8J6M7yf4i8BX0SOYf3TtOLdTVKKqxDoPKm8ZK-srQLm8DNyhvAv9tVzTgQ.11TuwI9FQiXiannx1MWrQQTbouhYjeY_Rq356ZCFORE&dib_tag=se&keywords=buck+converters&qid=1793003385&sprefix=buck+converter%2Caps%2C330&sr=8-3
   c. **Price – 2(240) - 480 Rs**

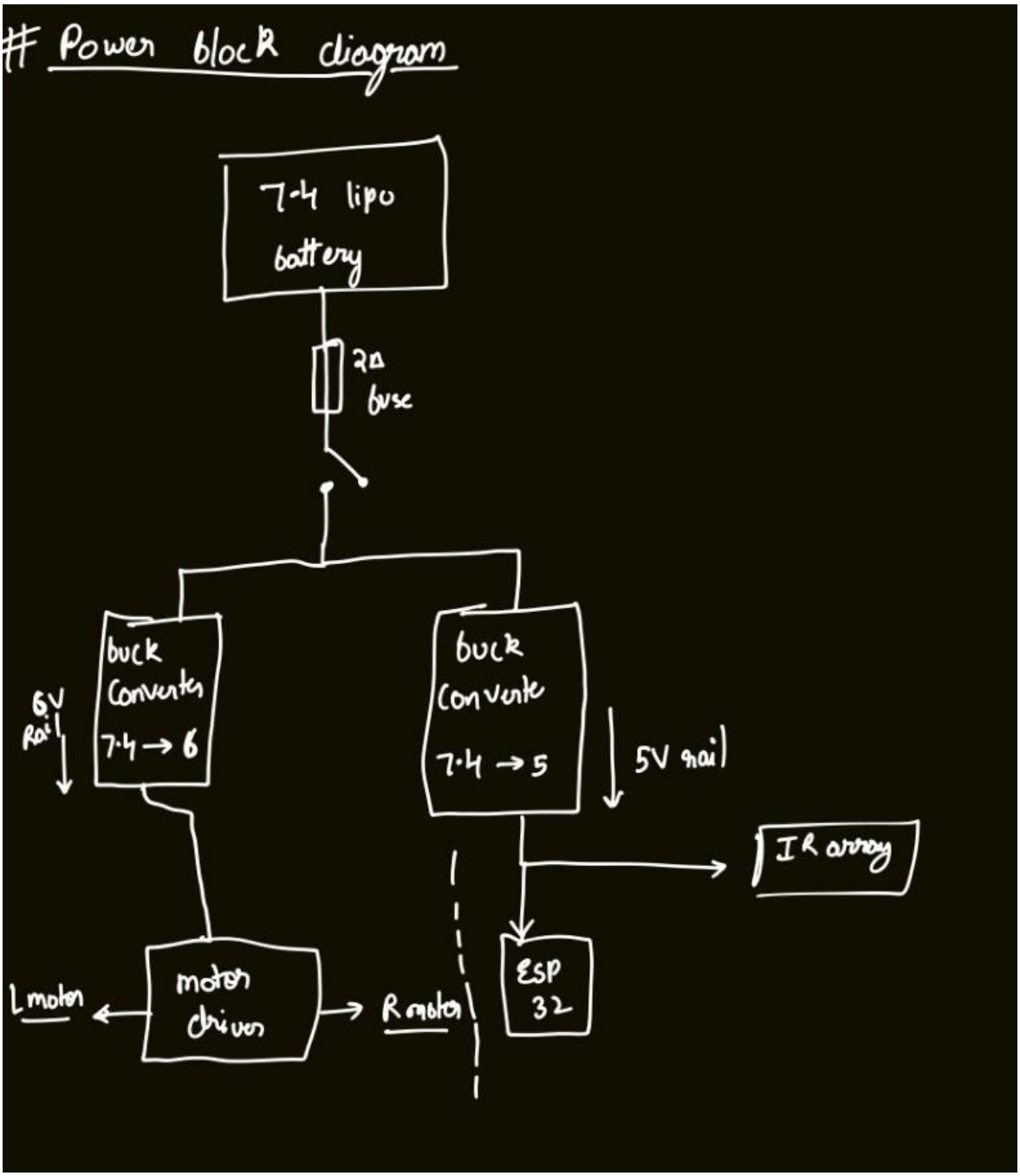
6. **3A Fuse ..**
    a. **Link - https://www.amazon.in/Ovicart-Glass-Fuse-Holder-3A/dp/B09X7B8BM8**
    b. Price – 180 Rs
    c. Reasoning is given where i calculated the max curent draw
7. And ye almost me bhul hi gaya(XT60 connector female and male both)
    a. Link - https://www.amazon.in/Universal-Hub-Pair-XT60-Connector/dp/B0GSBNDQ3L/525-8887240-1156925?pd_rd_w=9DD4Z&content-id=amzn1.sym.d1406b44-aa69-47e4-9270-f613e12d52dc&pf_rd_p=d1406b44-aa69-47e4-9270-f613e12d52dc&pf_rd_r=5NN851TSMB7TNE62N8GB&pd_rd_wg=lUUSA&pd_rd_r=adfcad74-263f-4b85-9296-ea022158de86&pd_rd_i=B0GSBNDQ3L&psc=1
    b. Price – 90 Rs

Total ~ 5k + 1.6k (for the battery) = 6.6k

For Battrey Refer to the Power budget and also the powerblock diagram ..

Task 5 - Power budget and battery selection

Power Block diagram



Battery selection and justification

- We can use the 2S LiPo · 7.4 V · 2200 mAh · 80C battery
    - The main reason why we use lipo batterys is that they deliver high current for their size ..so something like the compact bot of line maze solving ..
    - And also the motors need 6v so its easier to convert 7.4 to 6 and also to 5v basically

- Link - https://www.amazon.in/Original-ABSD-7-4V-2200MAH-Battery/dp/B0GCBRRP1Y?crid=3VTJ8JDSBHGPD&dib=eyJ2IjoiMSJ9.SsgH_-TyyLoZD2CLUlbU0bpgmFQODpvPgVIsCbn-h0va5gWtLmbxCz97PjToHbCnClyJOUIQvBcGzsP3wF2qcVOAWIdjM_Fok_t9YcMLL0V_rLos2MWrSsnzWMsbsHgdEVd-BumfBdXxhihwtLxkucYq3hu3HXNgWjYf-AmXcBezcLUmYJgl0VZ7VcbEvBhqzUsyVmVo_-haakVrdB8Dt3ToiqXtFjcq6AcMqViDUyg.gu161wE9vl6iEiQq6nRAd14dQ8ro1RZQvzNQ2JiEYNQ&dib_tag=se&keywords=2S+LiPo+%C2%B7+7.4+V+%C2%B7+1000+mAh+%C2%B7+25C&nsdOptOutParam=true&qid=1792959097&sprefix=2s+lipo+7.4+v+1000+mah+25c%2Caps%2C370&sr=8-3
- Price – 1600 Rs

Current draw estimation

- Lets talk abt the ratings first then go into calculations
-

Component	Idle	Peak	Basis
ESP32 DevKit	~80 mA	~250 mA	The chip draws 20–68 mA depending on clock speed with the radio off, and 240 mA at Wi-Fi transmit peak. I added ~10 mA for the DevKit's USB-UART chip, LED and regulator (my estimate).
QTR-8RC array	~100 mA	100 mA	LED current is about 20–25 mA per string, giving just under 100 mA for the board. Pololu lists the supply current as 100 mA.
Left N20 motor (6 V)	60 mA	550 mA	One common encoder-equipped N20 lists 0.55 A stall and under 0.06 A no-load at 6 V. SparkFun's N20 pair lists 0.03 A no-load and 0.45 A stall. I used the higher figures.
Right N20 motor (6 V)	60 mA	550 mA	Same as above.
Encoders (both)	~10 mA	~20 mA	Vendors only state a 3.3 V or 5 V supply, not a current. I assumed 5–10 mA each for the Hall sensors and pull-ups.
TB6612FNG	1.5 mA	2.2 mA	Logic current is 1.1–1.5 mA typical and up to 2.2 mA max, and standby is 1 µA max.
Buck converters (×2)	~10 mA	see Step 3	The LM2596 has about 5 mA quiescent current and 80% efficiency at 5 V out, 12 V in, 3 A.

- Summin the currents for rails .. there wll be 2 rails ..5v and 6v rails so

Rail	Idle	Peak
5 V logic (ESP32, QTR, encoders, TB6612 logic)	~192 mA	~372 mA
6 V motors (2 × N20)	120 mA	1,100 mA

- The worst-case total is about 1.5 A (just add the peak currents across all rails)

- But The max current that the battery i selected can give abt 176 A so it might be so that (say in a short more current than the 1.5 A is drawn in fact very high current may be drawn) in that case to protect my comopnets i need a fuse so we need a 3A fuse ....

Runtime calculation

- We are using a 2s 2200 mAh 80c Battery .. so we cannot use 100% of the battery say we use 75% i.e 1650 mAh and coz of the 80c we get 80*2200mAh = 176 A so the battery can supply this much current basically
- Suppose the typical driving and everything works normally and there is not stall and shii so then power of 5v rail is (0.4 + 0.5 + 0.06 ~ 1W) and the 6v rail power is (0.36 + 0.36 ~ 0.72) so total approximately is 1.7W
- So since the battery is 2200 mAh and 7.4 the total energy it has is 2.2* 7.4 but we considering only 75% so 2.2* 7.4(0.75) = 4.7 so we draw approx 1.7 W ideally so approx the car should work for ~ 7h ideally

Cell chemistry justification

-

Property	Li-ion (18650 cells)	LiPo (RC pouch pack)
Energy density	200–265 Wh/kg at cell level	150–200 Wh/kg at pack level
Max discharge rate	2–10C for standard cells	20–120C for RC-grade packs
Cycle life	500–1000 cycles	150–500 cycles
Self-discharge	1–3% per month	2–5% per month
Safety	The metal case makes it more robust to physical damage; LiPo has a medium rating because of puffing.	Soft pouch, so it can swell and be punctured. It usually has no built-in protection.
Shape	Rigid cylinder, needs a holder	Flat and low-profile, easy to fit in a chassis Table of Contents +8

- Why im using a LiPo battery compered to L ion battery
    - It's flat and compact, so it sits low on a small chassis, and you don't need a cell holder.
    - It handles motor start-up spikes and stalls with enormous headroom.
    - Its 7 h typical runtime is more than enough for a robot run.
- The disadvantages
    - **It's oversized.** My worst case is 1.44 A, which is 0.65C on a 2.2 Ah pack, while an 80C pack can deliver up to 176 A. The rating and extra capacity are mostly wasted, and they add weight.
    - **Developing on the big pack and swapping later is going to be my strategy(if the battrey i chose is that bulky then we can change to a smaller one after everything else works).** It gives long test sessions without recharging. It also has low internal resistance

Task 6 — Electronics & Hardware

Wiring diagram

- Mene jo bhi components use kia saare ek saaath koi online software me nahi the so ill give a circuit table and a rough hand drawn diagram ...

1. Main power path

#	From	To	Notes
1	Battery + (XT60)	Fuse in	Fuse goes in the **positive** lead only: 3 A slow-blow, 32 V rated.
2	Fuse out	Power switch terminal 1	Switch must be connected properly. (bohot baar ulta laga dia he mane)
3	Switch terminal 2	yaha +ve of battery split into 2(say W1 and W2	Splice that feeds both bucks.
4	W1	Buck A (5 V) IN+	
5	W2	Buck B (6 V) IN+	
6	Battery −	GND star point	The star point is Buck B's IN−
7	Battery JST-XH balance lead	Not connected	Use it only for charging(ofc)

2. Buck converters

#	From	To	Notes
10	Buck A OUT+	+5V rail	Set to 5.00 V by using a multimeter and also no load
11	Buck A OUT−	GND of 5v rail	
12	Buck B OUT+	+6V rail	Set to 6.00 V with no load first.
13	Buck B OUT−	GND of 6v rail	Short, thick wire, because this is the motor return path.

3. Power to the ESP32 and QTR (5 V and 3V3)

#	From	To	Notes
14	+5V rail	ESP32 VIN (or 5V)	
15	+5V rail	QTR-8RC VCC	Parallel with the ESP32. The QTR operates from 3.3–5 V.
16	GND of 5v rail	ESP32 GND	
17	GND of 5v rail	QTR-8RC GND	
18	ESP32 3V3	TB6612 VCC	

19	ESP32 3V3	TB6612 STBY	The TB6612FNG IC features built-in power management to conserve energy and prevent uncommanded motor behavior. So basically giving it 3.3 means the driver is always “on”
20	ESP32 3V3	Left encoder VCC	We want to map the maze also na …..
21	ESP32 3V3	Right encoder VCC	

4. Motor driver power and outputs

#	From	To	Notes
22	+6V rail	TB6612 VM	Add 10 µF plus 0.1 µF at the pin, and optionally 100–470 µF bulk.
23	GND of 6v rail	TB6612 GND (both pins)	Connect both ground pins.
24	TB6612 AO1	Left motor M1	Swap M1 and M2 if the direction is wrong.
25	TB6612 AO2	Left motor M2	
26	TB6612 BO1	Right motor M1	
27	TB6612 BO2	Right motor M2	

5. Signals: ESP32 GPIO map

- I picked pins that avoid the strapping pins (0, 2, 5, 12, 15) and the flash pins (6–11).
    - STRAPPING PINS (something i also learned now only!!)
        - Strapping pins" are special pins that the ESP32 reads their voltage level (HIGH or LOW) tells the internal processor how to start up
        - If you connect a motor driver input, sensor, or pull-up/pull-down resistor to one of these pins, that external component might force the pin HIGH or LOW during boot.
        - The ESP32 gets confused during startup and enters a "bootloop" or refuses to run your code at all until you unplug the pin.
    - Flash Pins
        - GPIO pins 6, 7, 8, 9, 10, and 11 are hardwired on the PCB directly between the ESP32 processor and that flash chip. They constantly transfer data back and forth while the robot runs.
        - If you try to use GPIO 6–11 as regular inputs or outputs for your motor driver, sensors, or encoders, you break the connection between the CPU and its memory.
        - The ESP32 instantly crashes or freezes, because it can no longer read the instructions of your program
- So avaoiding all the above

Device pin	ESP32 GPIO	Notes
TB6612 PWMA	25	Left motor speed
TB6612 AIN1	26	Left motor direction
TB6612 AIN2	27	Left motor direction
TB6612 PWMB	23	Right motor speed

TB6612 BIN1	18	Right motor direction
TB6612 BIN2	19	Right motor direction
Left encoder C1 / C2	34 / 35	Input-only pins, which suit encoders. They have no internal pull-ups.
Right encoder C1 / C2	36 / 39	Same as above.
Left and right encoder GND	GND star	Keep this separate from the motor's M2 pin.
QTR-8RC OUT1–OUT8	4, 13, 14, 16, 17, 21, 22, 32	One GPIO each. Read them as timed pulses.
QTR-8RC LEDON	33	The LEDs are on by default, and pulling this pin low turns them off. You can leave it unconnected.

Troubelshooting

- Voltage drop in the power supply
    - When the motors start, they can draw a large startup current. If the battery, wiring or buck converter cannot supply that current, the voltage may drop enough to reset the ESP32
    - Solving it -
        - Observe the 5 V rail during startup using a multimeter, if the 5 V rail drops sharply at the moment of reset, investigate the battery voltage, buck converter's current capability, wiring and connections first (ye mere saath hua tha utkarsh me .. wo car mera comptetion ke pehle tak sab sahi chal rha tha comptetion me disconnect hona shuru kar dia .. i believe wo voltage drop ke vajah se hi ho rha tha and mene vo esp ke ke vin and gnd pe capicator laga ke try kia but still ho rha tha .. still dont know ke wo kaise fix karna 🥀🥀🥀 ....)

- Poor grounding or loose connections
    - Ye esily fixable he you jus need to see ki koi connctions loose toh anhi he and thats it .. very common but also very easily fixable

Task 7 — Wireless Sensor Diagnostics

Protocol selection

- What we will do is compere all the 3 and then from the table we see what benifits us best in our case
-

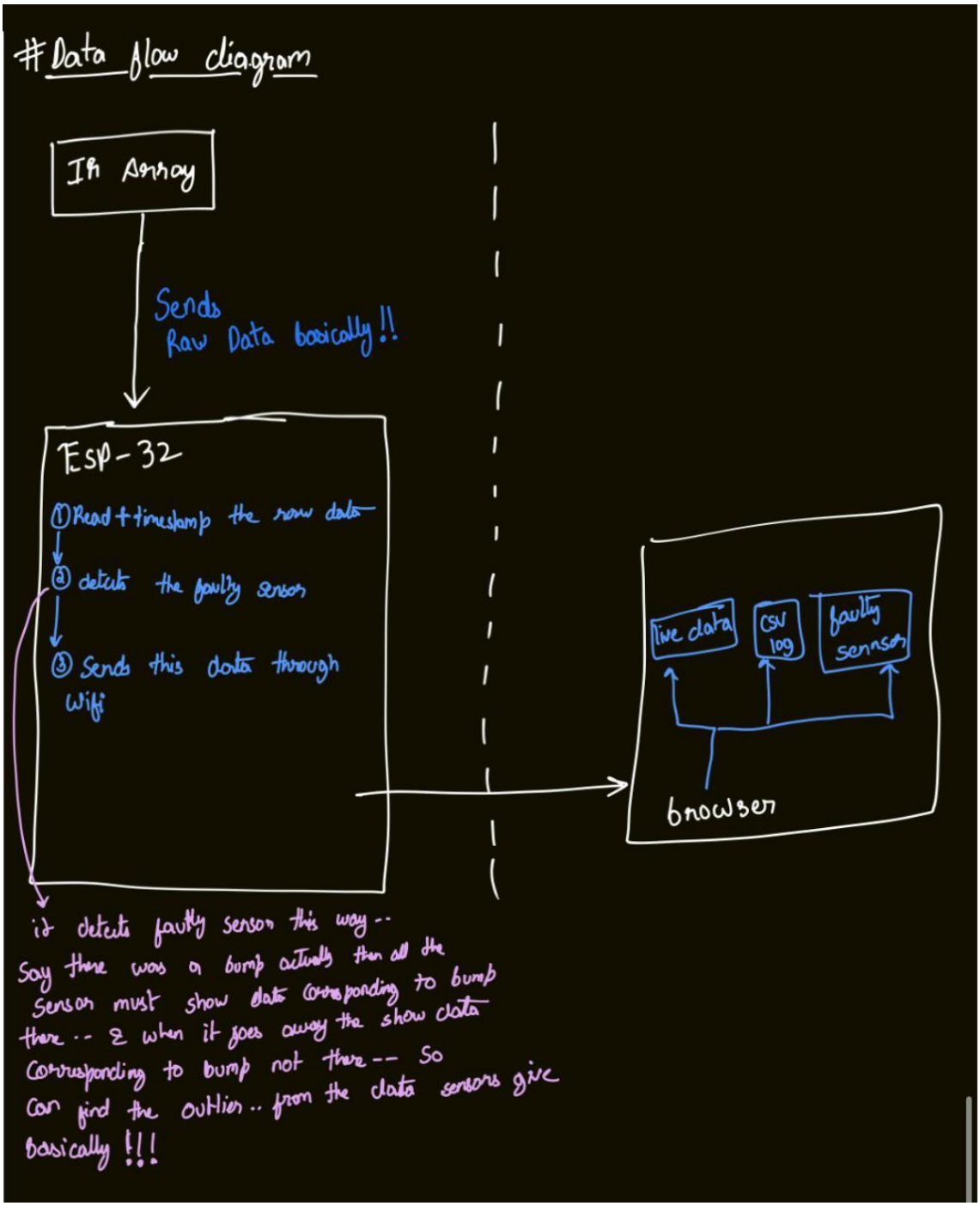
Criterion	Wi-Fi	Classic BT (SPP)	BLE
Latency	Low, but jitter is not so even basically	Low and steady	Low, but small packets and fixed intervals
Power	Highest, noisy bursts	Moderate	Lowest
Ease	Medium (needs AP, IP, client)	**Easiest**	Hardest (GATT setup)
Data rate vs. need	Far more than needed	**Enough** (~3 KB/s needed)	Enough, but must fragment data
Client	Browser or custom app	**Any serial terminal**	BLE app

- So what we will use is the WI-FI
    - **Latency**
        - Sensor glitches from bumps last only a few milliseconds, so I need data to arrive continuously and at a steady pace.
        - In Wifi mode, the debugger's phone or PC is the only device on the network. There is no shared traffic and no router in the path, so delay is low (roughly 5-20 ms) and steady.
        - We can apparently use a WebSocket so that as soon as the sensors get a reading we can ask the esp to push it to be on the web page ..
        - Classic Bluetooth SPP is similar in latency, so it has no real advantage here.
        - BLE sends small packets at fixed intervals, so data arrives in bursts and a short glitch can be smeared or missed.
    - **Power**
        - BLE uses the least power, but this is not a deciding factor. The motors draw far more than any BLE.
        - Wi-Fi draws the most radio current, in bursts of a few hundred mA. This is well within the 5 V rail budget (0.37 A maximum upar calculate kia he), and the motors run on their own 6 V buck, so the bursts will not cause a meaningful voltage drop i hbelieve. A capacitor near the ESP32's 5 V input should cover any small baxodi in voltage i believe.
        - So power ko dekh ke kisi cheez ko rule out nahi karenge coz almos saari hi similar he
    - **Ease of implementation**
        - The ESP32 hosts a small web page and a WebSocket server. Any phone or PC with a browser can connect, so no app has to be installed.
        - Writing the page (a live plot of the 8 sensors, with the faulty channel highlighted) is a small task, and the same page also provides the logging (a CSV download button).

- It works on Android, iOS, Windows and Linux alike. Classic Bluetooth SPP does not work on iPhones.
- Classic Bluetooth is simple for raw text, but it still needs a serial terminal app or a Python script to plot the data. That is about the same effort as the web page and gives a worse display.
- BLE needs GATT services and characteristics and a BLE app, which is the hardest of the three

- **4.Utkarsh Reasons**
  - Meri utkarsh me gaddi wifi baaar baar disconnect ho rhi thi .. ab baat ego pe aagyi he meri so issbar same problem aayigi toh me fix karke hi rahunga bc 😤😤🥀🥀

System architecture

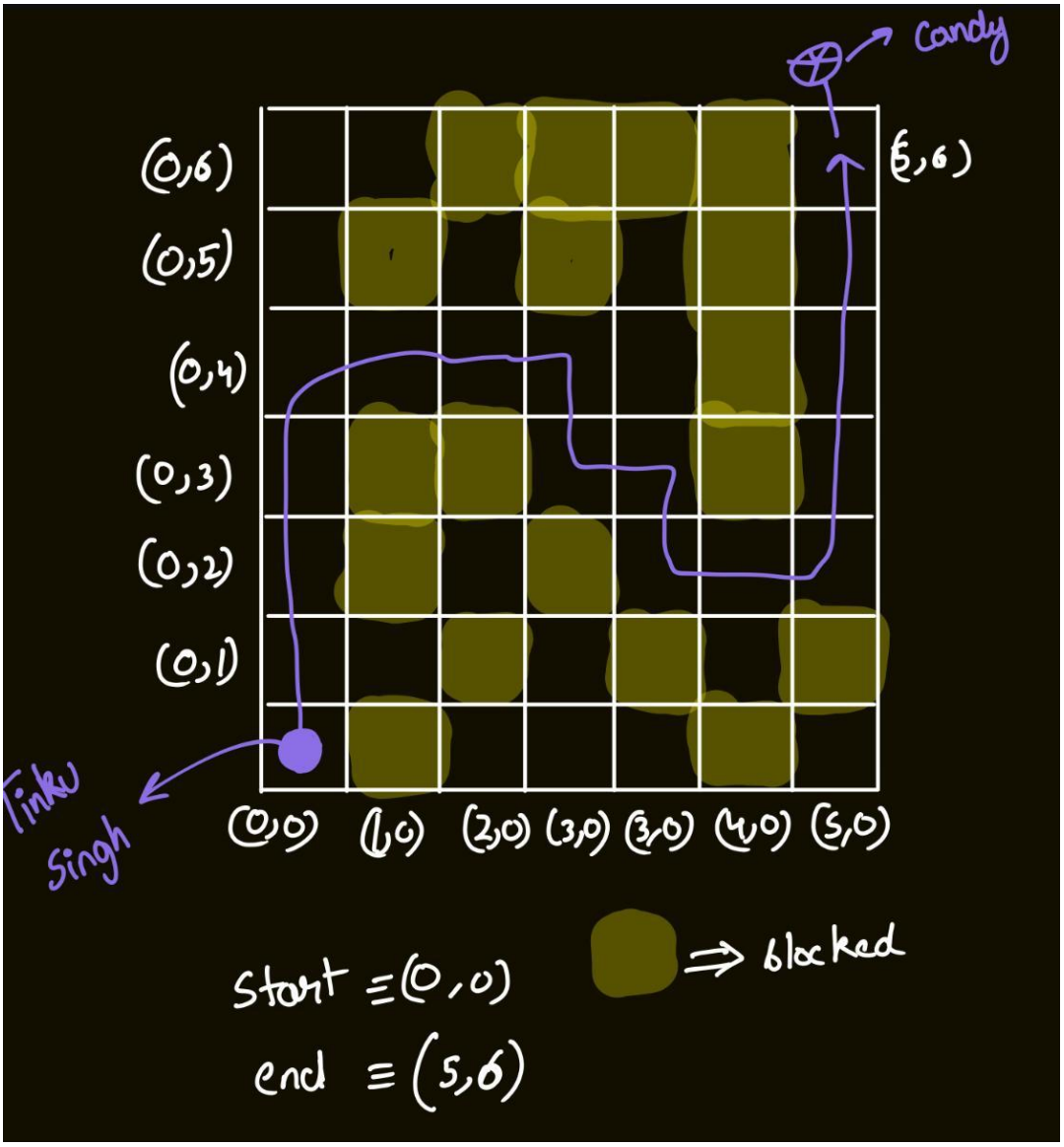


Task 8 — Maze-Solving Algorithms

Comparison

We will use DFS algorithm for this

- DFS explained
    - **Main idea**
        - Go as deep as possible down one path. When you hit a dead end, go back to the last place where you had another choice, and try that instead. Keep doing this until you reach the goal.
    - **What the robot must remember**
        - **Visited cells:** a grid marking which cells it has already been in.
        - **Stack (the path so far):** a list of the cells it passed through, in order. The newest is on top. This is how it knows where to backtrack.
        - **Current position and heading:** which cell it is in, and which way it faces (from the encoders).
        - **Walls seen:** which sides of each cell are open or blocked (from the sensors).
    - **Steps**
        - Start at the start cell. Mark it visited and push it on the stack.
        - Look at the neighbours: which sides are open (no wall) **and** not yet visited?
        - **If there is one or more:** pick one using a fixed order (for example: straight, left, right). Move into it, mark it visited, push it on the stack.
        - **If there is none (dead end, or all neighbours visited):** pop the top cell off the stack and move back to the cell below it. This is **backtracking**.
        - Repeat from step 2 until the current cell is the goal.
        - When the goal is reached, the stack holds the path from start to goal.
    - Understanding with an example …
        - **Rule:** a **node** is any place where the robot must decide something or where a path ends. An **edge** is the stretch of path between two nodes.
        - **Nodes:** the start, the goal, dead ends, and junctions (cells with 3 or more open sides).
        - **Edges:** corridors between nodes. The edge's length is the number of cell steps, or the encoder ticks the robot counts (yaad aaya mene q bola tha ki hame wo n20 motor chaye jisme encoders ho ..?).
        - **Not nodes:** cells in a plain straight corridor or a simple corner. There is only one way to go, so nothing is decided. They are just part of an edge.
        - Consider this maze

- 

- We need to give it a fixed arbitary direction prority say up > right > down > left
- The robot starts at (0,0).
  - It marks (0,0) as visited.
  - It checks its neighbouring cells.
  - Suppose moving up to (0,1) is possible, while moving right is blocked.
  - The robot moves to (0,1) and marks it as visited.
  - At (0,1), suppose the next available cell is (0,2). The robot moves there.
  - It continues moving through available, unvisited cells, such as (0,3) and (0,4).
  - At this stage, the robot is exploring one path without considering whether another path might be shorter.
- Suppose the robot reaches (0,4). From this cell, two neighbouring cells are available:
  - Up: (0,5)
  - Right: (1,4)
  - Since the direction priority is Up, Right, Down, Left, the robot chooses (0,5) first.

        - It does not compare the two paths or calculate which one is better. It simply follows its direction priority and explores the upward path first.
    - Suppose the robot follows the path: (0,4) → (0,5) → (0,6) → (1,6)
        - Now suppose (1,6) has no open, unvisited neighbours.
        - The robot has reached a dead end.
        - DFS handles this situation through a process called backtracking.
        - The robot retraces its path:
        - (1,6) → (0,6) → (0,5) → (0,4)
        - It returns to (0,4), the previous junction.
        - The robot remembers that it has already explored the upward path. Therefore, it does not explore that path again.
        - Instead, it checks for another available neighbour. Since (1,4) is still open and unvisited, the robot moves right to (1,4) and continues exploring.
        - This is the key idea behind DFS: when a path leads to a dead end, the robot goes back to the most recent position where an unexplored path is available.
    - The robot continues repeating the following process:
        - Check the neighbouring cells in the chosen direction order.
        - Move to an open, unvisited neighbour.
        - Mark the new cell as visited and add it to the current path.
        - If no open, unvisited neighbour exists, backtrack.
        - Repeat until the destination (5,6) is reached.
        - The robot does not need to know the entire maze beforehand. It discovers the available paths while exploring.

Exploratory Traversal vs Optimal-Route Replay

- **Exploratory Traversal (Unknown Maze):**
    - The robot does not know the maze in advance. It explores the available paths, detects junctions and dead ends, marks visited cells, and uses DFS with backtracking to discover the maze. It may need to explore multiple paths before reaching the destination.
- **Optimal-Route Replay (Known Map):**
    - Once the maze has been mapped, the robot calculates the shortest route from the start to the destination and follows that route again. It no longer needs to explore alternative paths or perform backtracking.
- **Why is replay simpler?**
    - During replay, the robot already knows which path to take at each junction. Therefore, it only needs to follow the predetermined route and make the required turns, rather than repeatedly exploring and evaluating different paths. This simplifies the decision-making logic and can reduce the time needed to reach the destination.